

Group 6: High-Level Test Plan

Sidharth Krishnaswamy, Nishit Oberoi, Edmond Pan, Kyle Kawasuna Saiki, Lu Maw Tun, Katia Yarkov

I. Introduction

Testing is an essential part of software development and is critical to delivering a dependable implementation of OFS's food delivery service. This document presents our high-level testing plan, based on the practices described in *Engineering Software Products* by Ian Sommerville this document outlines a staged strategy of unit, feature, and system testing.

II. System Under Test + Assumptions

The System Under Test (SUT) is OFS, a full-stack web application that supports online ordering and delivery of organic groceries in Downtown San Jose. Customers can register/login, browse and search products, add items to a cart, and place orders for delivery. The system calculates total cart weight and applies delivery pricing rules, creates and stores orders in a relational database, updates inventory after purchase, and provides order status tracking. OFS also includes an employee/manager dashboard for inventory management and order/report viewing.

SUT includes:

- React based customer web frontend
- React based admin web frontend
- FastAPI backend with REST endpoints
- MySQL Database

Assumptions:

- Robotic delivery is simulated: Robots are represented as software agents that receive a delivery batch and a Google Maps route; no physical robotics hardware/sensors are tested.
- Google Maps Dependency: Google Maps API is available and accurate throughout testing and deployment
- Payments + notifications are mocked: Checkout uses a mock payment behavior and mock email/SMS confirmations. No real financial transactions or real customer messages occur.
- Supported browsers for testing: The web client will be tested on Chrome and Firefox.
- Mobile client is unsupported: The project description for OFS emphasizes a web-based customer site, but does not require a mobile client.

III. Testing Scope

Customer web application (REACT)

- Proper authentication and authorization of users and their roles on the app (JWT)
- Accurate and seamless product browsing and filtering
- Cart functionality and proper weight calculations
- Accurate delivery pricing logic (item pricing calculations, tax?, delivery fee addition according to weight)
- Checkout and order creation
- Proper inventory updates after purchases server-wide, especially when multiple users are checking out
- Admin dashboard operations: properly view, add, remove stock
- Order tracking and status updates during delivery
- Google Maps API integration behavior
- Overall application security, performance, and load testing

Backend Rest API (FastAPI)

- Authentication + access control
- Product catalog endpoints
- Inventory updates
- Routing and messages to self driving fleet

Database (MySQL)

- Secure storage of users data
- Data Integrity Checks

IV. Testing Goals

- Ensure overall correctness from all functional requirements
- Ensure proper and secure authentication and role-based access across service
- Update and confirm accurate inventory and order data consistency
- Detect edge-case failures and interaction defects
- Validate and sustain performance under realistic user and data load
- Ensure proper usability and accessibility for end all users
- Find defects early
- Prove that system is fit for purpose
- Validate business logic
- Reduce risk of integration with modules

V. Functional Testing

Unit Testing

Unit tests will be designed using equivalence partitions so that each distinct class of inputs (both valid and invalid) is exercised, with special emphasis on boundary values where off-by-one defects commonly occur. We will deliberately force error conditions to verify that units detect and handle incorrect inputs predictably, and we will always include null/empty/zero/negative cases to ensure robustness.

- Authentication Validation: Validate email format and password rules, and verify credential checking handles valid login, wrong password, unknown email, empty/null, and overly long inputs.
- Product / Search Query Helpers: Ensure search/filter normalization returns correct matches and that empty/whitespace/not-found queries return a safe empty result
- Cart Validation + Calculations: Validate quantities within stock limits and reject 0/negative/overstock/huge value
- Delivery Fee Calculation: Confirm delivery fee is computed correctly across weight partitions with boundary tests
- Inventory Update Logic: Verify stock checks prevent overselling and inventory decrements correctly while enforcing invariants like “inventory never negative”
- Delivery Scheduling Constraints: Test batching logic respects robot constraints (≤ 10 orders and ≤ 200 lbs per trip)
- Route Service Wrapper : Validate correct parsing/formatting of routes and ETA when Maps succeeds and graceful error handling (no crash) if google maps fails
- One is different: Check that the first and last order of the day are successful

Feature Testing

- User registration and login
 - Goals:
 - Ensure users can successfully create accounts and authenticate securely
 - Ensure role-based behaviors (admins and users) are enforced accurately
 - Expectations:
 - Registration creates a new user record into MySQL database with securely hashed password
 - Login with valid credentials returns success and sets JWT in httponly cookie
 - Invalid credentials return safe error message
 - Administrators are redirected to admin UI, general users to customer UI
 - User logout clears cookie and prevents access to protected resources
 - Tests

- Unit: password hashing/verification; JWT creation/verification
 - Integration: POST api/auth/register, POST api/auth/login, GET api/auth/me, POST api/auth/logout
 - End-to-End: register → login → redirected based on role; attempt admin page as user → redirect/forbidden
- Product listing and search
 - Goals:
 - Ensure users can browse the catalog
 - Ensure users can search/filter products accurately
 - Expectations:
 - Product list loads consistently with correct names, descriptions, price, weights, stock
 - Search function returns relevant products and handles empty/no results smoothly
 - Filtering/sorting behaves deterministically
 - Out-of-stock items are clearly indicated and cannot be purchased
 - Tests
 - Unit: Frontend search UI, rendering
 - Integration: GET api/products, GET api/products?filters=, GET api/products/productID=
 - End-to-End: search/filter for known product → appears; search nonsense → no results
- Cart management
 - Goals:
 - Ensure cart operations behave correctly and maintain consistent totals
 - Expectations:
 - Add/remove/update quantity updates cart state correctly
 - Quantity cannot be negative or exceed available stock
 - Cart total items, total price, and total weight are updated immediately and correctly
 - Tests:
 - Unit: cart reducer/state logic; quantity validation
 - Integration: POST /cart/add, PATCH /cart/item, DELETE /cart/item, GET /cart/summary
 - E2E: add items → totals update → remove → totals update
- Weight-based pricing
 - Goals:
 - Ensure delivery fees and pricing rules based on total cart weight are correct and consistent
 - Expectations:

- Weight calculation equals sum (productWeight * quantity)
 - Delivery pricing matches defined rules (< 20lbs. Free delivery, otherwise \$10 delivery fee)
 - No negative or undefined fee values
 - Pricing updates when cart change
- Tests:
 - Unit (backend): pure pricing function tests with boundary weights (just below/at/above threshold)
 - Integration: checkout endpoint, returns correct breakdown
 - E2E: modify cart weight (add/remove items)→ fee changes accordingly
- Checkout flow
 - Goals:
 - Ensure checkout proceeds correctly from cart → confirmation using mocked payment behavior
 - Expectations:
 - Checkout validates cart is non-empty and items are in stock
 - Mock payment success produces an order; failure shows error and no order created.
 - Double-submit protections prevent duplicate orders
 - User receives confirmation (mocked notification event recorded)
 - Tests:
 - Unit: checkout validation logic; idempotency key behavior (if implemented).
 - Integration: POST /checkout with mock payment success/failure,
 - E2E: checkout → confirmation page; simulate payment failure → remain in checkout with error.
- Order creation and storage
 - Goals:
 - Ensure orders are created correctly and stored consistently into database
 - Expectations:
 - Successful checkout creates exactly one order record with correct line items, totals, fees, timestamps.
 - Order is linked to user account
 - Database commits occur atomically with inventory updates (no partial states)
 - Order IDs are unique; order history is accurate
 - Sent to Delivery Scheduling Service
 - Tests:
 - Unit: order total computation and line item formation.

- Integration: POST /orders (or checkout endpoint) verifies DB rows created correctly.
 - E2E: place order → view order history includes it with correct details
- Inventory management
 - Goals:
 - Ensure inventory is accurate, protected by role-based access control, and updated correctly after purchases and admin edits
 - Expectations:
 - Only admin users can access inventory endpoints/pages
 - Inventory decrements after successful order creation
 - Inventory never drops below zero; concurrent purchases handle race conditions safely
 - Admin edits persist and immediately reflect in product listing
 - Tests:
 - Unit: inventory decrement function and validation.
 - Integration: PATCH /admin/inventory/{id}; concurrency tests (two orders racing on low stock)
 - E2E: admin updates stock → customer sees change; user tries admin endpoint → 403 (lack perms)
- Order status tracking
 - Goals:
 - Ensure users and admins can view accurate status and that status transitions are valid
 - Expectations:
 - Orders move through valid states (Processing→ Out for Delivery → Delivered)
 - Customers can view their own order status but not others'
 - Admin/robot agent can update status according to rules
 - Status history timestamps/etas are stored and displayed correctly
 - Tests:
 - Unit: status transition validation rules.
 - Integration: GET /orders/{orderId}/status, GET /orders/{orderId}/location , PATCH /admin/orders/{id}/status.
 - E2E: place order → status visible; admin changes status → customer sees updated status

Interaction Tests

- Test the full authentication chain to confirm features cooperate correctly across UI, API, and database, including successful login establishing an authenticated session, correct role resolution, correct routing to customer or admin views, and strict enforcement of role-based access to protected pages and endpoints.
- Test inventory management consistency across components by confirming that admin updates to stock or product data persist in MySQL, propagate through backend product APIs, and immediately appear correctly in the customer catalog and product detail views, including correct out-of-stock display and purchase blocking.
- Test the purchase and fulfillment pipeline across components by confirming that browse and cart actions carry correct quantities

Usefulness Tests

We will use a non-developer participant to interact with our app from login to checkout while we observe time on task, number of clicks, eye tracking, and have them write a written review afterwards

- Use a non-developer participant recovery from common errors such as wrong password, out-of-stock items, or checkout validation messages.
- Use a non-developer participant to find specific items using category browsing, keyword search, and filters, and evaluate whether results feel relevant and predictable.
- Use a non-developer participant acting as an admin to perform common inventory tasks such as adjusting stock, adding a product, and correcting mistakes.

System Testing

System testing will evaluate OFS as a complete, integrated system with a focus on user-perceived performance in Chrome and Firefox. We will run repeatable end-to-end user pathways such as browsing the catalog, searching and filtering products, adding items to the cart, and completing checkout, and we will time key actions including page loads and interaction response times in both browsers to confirm the system remains responsive and consistent across the required client environments. We will also assess performance and behavior under degraded network conditions by temporarily disconnecting from the internet to verify the application fails gracefully, displays clear error states, and recovers cleanly once connectivity is restored.

VI. Test Automation

We will automate unit testing using Jest for the React frontend and pytest for the Python backend, writing executable tests in an arrange–action–assert style so they can be rerun consistently for regression testing. For database-related logic, we will populate a dedicated test database with realistic dummy values and use this seeded data to validate SQL queries and database interactions under repeatable conditions.

VII. User Testing

Alpha testing will be done early with a small set of testers working directly with the team while we observe them completing core OFS tasks such as search, cart management, and checkout to identify major usability issues and missing functionality. Beta testing will be done later by giving a near-complete build to testers to use independently in a realistic environment and collecting feedback on usability, clarity of errors, and overall usefulness.

VIII. Code Reviews

Before any update is pushed to GitHub, another team member will review the code and associated tests to catch defects, misunderstandings, and missed edge cases. This review process helps ensure changes meet requirements, follow team standards, and do not introduce regressions before they are merged.

IX. Load Testing

For load testing, we will simulate multiple users interacting with the site at the same time to evaluate response time and overall stability under increased demand. We will specifically look for concurrency issues and race conditions.